

opentitan硬件HW开发【步骤】

https://opentitan.org/book/doc/getting_started/index.html#step-8-additional-resources
关键文档导引

https://opentitan.org/book/doc/contributing/github_notes.html#github-notes
github 相关

https://opentitan.org/book/doc/contributing/bazel_notes.html
bazel命令相关

添加新IP到SOC

一、创建文件夹 `mkdir hw/ip/sm4` 再建data和rtl data下建sm4.hjson(参考SM4)

写IP的hjson文件，在里面定义寄存器和其他各种信号

二、`util/regtool.py -r \-t hw/ip/sm4/rtl/ hw/ip/sm4/data/sm4.hjson` 生成寄存器文件到IP的RTL文件夹
`util/regtool.py -D -o hw/ip/sm4.h hw/ip/sm4/data/sm4.hjson` 生成寄存器c头文件

三、更改顶层/top_earlgrey/data/top_earlgrey.hjson 文件，将IP连到顶层模块（参考SM4）

四、添加IP到hw/Makefile文件

五、添加到总线，改hw/top_earlgrey/data/xbar_main.hjson（参考SM4）
注意两处要改

六、编写 hw/ip/sm4/sm4.core （参考SM4） （给fusesoc构建SOC用）

sv中包含的文件后面要跟 {is_include_file: true}

七、将IP添加到hw/top_earlgrey/top_earlgrey.core

八、make -C \$REPO_TOP/hw top make update顶层

make -C \$REPO_TOP/hw all

九、构建verilator测试

bazel test //sw/device/my_tests/sm4:sm4_smoketest_sim_verilator

(十)、fusesoc --cores-root . run --flag=fileset_top --target=synth --setup
lowrisc:systems:chip_earlgrey_\${BOARD}

构建FPGA工程（复制源代码文件到暂存区）

source lowrisc_systems_chip_earlgrey_cw310_0.1.tcl 从TCL 打开工程。

在定义寄存器和HW连接时的一些注意事项：

1、关于硬件写寄存器

在hw2reg中d和de是对hw端的。在reg2hw中，硬件写的寄存器的q是对总线（软件端）的。

hw若想更新寄存器的数据，就要把数据放到d上，同时de拉高，下个时钟相应的q就会更新。

如果hw的有效输出数据无法长期保持，de就要采用脉冲的方式（用有效标志驱动de），这样就只会在数据有效时更新reg，软件读到的是有效数据。

2、关于IP寄存器基地址

IP间基地址要相隔2000

<https://opentitan.org/book/util/reggen/index.html#register-definitions-per-type>

name	direction	description
we	input	Write enable from the bus interface, scalar
wd[]	input	Write data from the bus interface, size == field size
qs[]	output	Output to read response data mux, size == field_size. This is typically the same as q[] except for hwext registers.
de	input	Hardware data enable from peripheral logic, scalar, should be set when the hardware wishes to update the register field
d[]	input	Hardware data from peripheral logic, size == field size
qe	output	Output register enable, scalar, true when bus interface writes to subreg
q[]	output	Output register content, size == field size. This output typically goes to both the hardware bundle output q as well as the software read response mux output qs[] .
re	output	Indicates to hwext registers that the bus is reading the register.

关于IP间信号的创建

- 1、IP hjson
- 2、top hjson

Defining the inter-module signal

The example configuration file above specifies two cases of inter-module signals, `msg_fifo` and `suspend`.

Attribute	Mand/Opt	Description
name	mandatory	<code>name</code> attribute specifies the port name of the inter-module signal. If the type is <code>req_rsp</code> , it indicates the peripheral has <code>name_req</code> , <code>name_rsp</code> ports (with <code>_i</code> and <code>_o</code> suffix)
struct	mandatory	The <code>struct</code> field defines the signal's data structure. The inter-module signal is generally bundled into <code>struct packed</code> type. This <code>struct</code> is used with <code>package</code> for topgen tool to define the signal. If the inter-module signal is <code>logic</code> type, <code>package</code> field can be omitted.
package	optional	
type	mandatory	There are two types of inter-module signal. <code>req_rsp</code> is a connection that a module sends requests and the other module returns with responses. <code>uni</code> is one-way signal, which can be used as a broadcasting signal or signals that don't need the response.
act	mandatory	<code>act</code> attribute pairs with the <code>type</code> . It specifies the input/output of the signal in the peripheral. <code>req_rsp</code> type can have <code>req</code> (requester) or <code>rsp</code> (responder) in <code>act</code> field. <code>uni</code> type can have <code>req</code> or <code>rcv</code> (receiver) in <code>act</code> .
width	optional	If <code>width</code> is not 1 or undefined, the port is defined as a vector of struct. It, then, can be connected to multiple peripherals. Currently, <code>logic</code> doesn't support the connection to multiple modules if <code>width</code> is not 1.

关于使用hjson进行规范的IP间信号的建立

sv头文件及其作用域的拓展

修改

chip_earlgrey_xxx.sv 要注意的地方

IP间信号

在IP Hjson 中用 `inter-signal-list` 属性定义
模块之间连接在顶层画己置文件中定义,

A 模块.V 怎么写:

① 用 struct 定义信号, 放在 IP-pkg.sv 的 package. IP-pkg 中

② 导入 pkg. 并用结构体定义 Input/output 接口类对象(信号, 加_i或_o后缀)
注: 若信号为 logic 类型, 可不作上面两步

B IP.Hjson 怎么写.

```
{
  struct: " " 定义信号的结构体[去掉_t字符]
  type: " " 有双向 req_rsp, 单向 uni. 尽量用单口
  name: " " 信号名, sv中接口定义要再加上_i或_o
  act: " " 标志, IP中此信号的方向, in为rcv, out为req[在单端口中]
  package: " " 结构体所在的包
}
```

注: 若要把信号(s)连到多个外设则加上 width 属性 eg: width: "3", 到3个
logic 不支持连到多个外设.

C top.Hjson 怎么写.

在 inter-module 下, 接着写.

```
'IP1.信号名name1': ['IP2.信号名name2', 'IP3.name3', ...],
IP1的信号, 连到了 { IP2的name2
                    IP3的name3
```

struct与act属性关系.

解决头文件路径问题? 试用 logic 类型.

PUT出来的RNG扩展成4位并异或与challenge. → 根为熵.
entropy-src. 火商源. PUT出来的直接接入.

附: 关于增加头文件及其他IP的引用

① 所有被视为主文件的都要有在其

IP₁.core 中: {is_include_file: true}
的标识.

② 一个IP若要用头文件中的 PKG
则要先包含, 再导入

③ 另一个IP若要用上一个IP的 PKG
则在 IP₂.core depend 域中包含IP₁.

关于修改 chip_earlgrey_xxx.sv
中的大模块的连接关系
在 Top_earlgrey.hjson 修改之后
make-c. 后, 手动修改检查
chip.sv

chip.sv 中的连接关系尽量

不云力

接上红: 用 .core 法, 用到的 package 包新
建一个 IP-Pkg.core [参照 SM4]

要用的 other IP 在自己的 core 中 depend
项加上, 在 IP 中导入后可用

关于chip_earlgrey层sv文件的编辑

完成top_earlgrey层的编辑后，手动修改芯片层。

https://opentitan.org/book/hw/top_earlgrey/doc/design/index.html?highlight=top_ea#rtl-implementation-notes